

✓ TOÁN ỨNG DỤNG VÀ THỐNG KÊ

Đồ án Linear Regression

Mục lục

- [Thông tin sinh viên](#)
- [Mô tả bài toán và giải thuật](#)
- [Chuẩn bị dữ liệu và cài đặt](#)
- [Câu a: Xây dựng mô hình với toàn bộ 11 đặc trưng](#)
- [Câu b: Tìm và sử dụng duy nhất 1 đặc trưng tốt nhất \(dùng Cross-Validation\)](#)
- [Câu c: Xây dựng một mô hình của riêng bạn cho kết quả tốt nhất](#)
- [Mô tả ý tưởng thực hiện](#)
- [Mô tả các hàm](#)

=====

PHẦN THÔNG TIN SINH VIÊN

=====

Họ và tên: Nguyễn Hải Đăng

Mã số sinh viên: 23127165

Lớp: 23CLC03

=====

✓ =====

PHẦN MÔ TẢ BÀI TOÁN VÀ GIẢI THUẬT

Mô tả Bài toán

Dự án này sử dụng tập dữ liệu `wine.csv` chứa thông tin đánh giá chất lượng của 1200 chai rượu vang theo thang điểm từ 1 đến 10, dựa trên 11 tính chất hóa học khác nhau.

Mục tiêu là xây dựng mô hình dự đoán chất lượng rượu vang sử dụng phương pháp hồi quy tuyến tính, với thuật toán OLS (Ordinary Least Squares):

- Sử dụng toàn bộ 11 đặc trưng để xây dựng mô hình.
- Sử dụng duy nhất một đặc trưng cho kết quả tốt nhất (áp dụng kỹ thuật Cross Validation).
- Đề xuất và triển khai một mô hình riêng cho kết quả tốt nhất.

Hồi quy tuyến tính và Phương pháp Bình phương tối thiểu (OLS)

Hồi quy tuyến tính là một thuật toán học máy có giám sát, được sử dụng để mô hình hóa mối quan hệ tuyến tính giữa một biến phụ thuộc (còn gọi là mục tiêu) và một hoặc nhiều biến độc lập (còn gọi là đặc trưng). Mô hình toán học của hồi quy tuyến tính có dạng:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n + \epsilon$$

Trong đó:

- y : Biến phụ thuộc (giá trị mục tiêu cần dự đoán).
- $x_1, x_2, \dots, x_n$: Các biến độc lập (các đặc trưng đầu vào).
- $\theta_0, \theta_1, \dots, \theta_n$: Các hệ số (hay trọng số) của mô hình. θ_0 được gọi là hệ số chặn (intercept).
- ϵ : Sai số ngẫu nhiên, biểu thị cho các yếu tố không thể giải thích được bởi mô hình.

Mục tiêu của hồi quy tuyến tính là tìm ra một bộ hệ số θ sao cho mô hình có thể dự đoán giá trị $\hat{y}$ (y-mũ) gần với giá trị thực tế y nhất.

Phương pháp Bình phương tối thiểu thông thường (Ordinary Least Squares - OLS)

Phương pháp **Bình phương tối thiểu thông thường** (OLS) là một trong những kỹ thuật phổ biến và nền tảng nhất để ước tính các hệ số của mô hình hồi quy tuyến tính. Nguyên tắc cốt lõi của OLS là tìm ra bộ hệ số θ sao cho tổng của bình phương các phần dư (sai số giữa giá trị thực và giá trị dự đoán) là nhỏ nhất.

Hàm mục tiêu

OLS hoạt động bằng cách tối thiểu hóa **Tổng bình phương các phần dư** (Residual Sum of Squares - RSS):

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\theta_0 + \theta_1 x_{i1} + \dots + \theta_p x_{ip}))^2$$

Trong dạng ma trận, hàm mục tiêu được biểu diễn gọn hơn:

$$RSS(\theta) = (y - X\theta)^T (y - X\theta) = \|y - X\theta\|_2^2$$

Công thức nghiệm (Normal Equation)

Bằng cách giải bài toán tối ưu hóa trên, ta có thể tìm được nghiệm giải tích chính xác cho vector hệ số θ . Công thức này được gọi là **Phương trình pháp tuyến** (Normal Equation):

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

Trong đó:

- θ : Vector các hệ số ước tính cần tìm.
- X : Ma trận thiết kế (design matrix), chứa các giá trị của biến độc lập (bao gồm một cột các số 1 cho hệ số chặn θ_0).
- X^T : Ma trận chuyển vị của X .
- $(X^T X)^{-1}$: Ma trận nghịch đảo của $X^T X$.
- y : Vector chứa các giá trị thực tế của biến phụ thuộc.

OLS là một phương pháp mạnh mẽ, hiệu quả và dễ diễn giải, làm nền tảng cho nhiều kỹ thuật phân tích thống kê và mô hình hóa phức tạp hơn.

Kiểm định chéo (Cross-Validation)

Để đánh giá độ chính xác của mô hình một cách khách quan và tránh tình trạng học vẹt (overfitting), chúng ta sử dụng kỹ thuật Kiểm định chéo K-Fold (K-Fold Cross-Validation).

1. **Chia dữ liệu**: Chia ngẫu nhiên tập dữ liệu thành K phần (fold) có kích thước bằng nhau.

2. **Lặp K lần**: Trong mỗi lần lặp:

- Chọn 1 phần làm tập kiểm tra (validation set).
- Sử dụng K-1 phần còn lại làm tập huấn luyện (training set).
- Huấn luyện mô hình trên tập huấn luyện để tìm ra bộ θ .
- Dùng mô hình vừa huấn luyện để dự đoán trên tập kiểm tra và tính sai số (ví dụ: MSE).

3. **Đánh giá**: Sai số cuối cùng của mô hình là giá trị trung bình của các sai số tính được trong K lần lặp. Kỹ thuật này cho phép mô hình được đánh giá trên tất cả các mẫu dữ liệu.

✓ =====

CHUẨN BỊ DỮ LIỆU VÀ CÀI ĐẶT

```
import pandas as pd
df = pd.read_csv('wine.csv')
```

df



	"quality"	"fixed acidity"	"volatile acidity"	"citric acid"	"residual sugar"	"chlorides"	"free sulfur dioxide"
0	5	7.4	0.700	0.00	1.9	0.076	11.0
1	5	7.8	0.880	0.00	2.6	0.098	25.0
2	5	7.8	0.760	0.04	2.3	0.092	15.0
3	6	11.2	0.280	0.56	1.9	0.075	17.0
4	5	7.4	0.700	0.00	1.9	0.076	11.0
...	...	...	...	...	...	...	...
1194	6	7.0	0.745	0.12	1.8	0.114	15.0
1195	6	6.2	0.430	0.22	1.8	0.078	21.0
1196	6	7.9	0.580	0.23	2.3	0.076	23.0
1197	6	7.7	0.570	0.21	1.5	0.069	4.0
1198	6	7.7	0.260	0.26	2.0	0.052	19.0

✓ =====

XÂY DỰNG MÔ HÌNH ĐÁNH GIÁ SỬ DỤNG PHƯƠNG PHÁP HỒI QUY TUYẾN TÍNH

=====

=====

✓ Câu a: Sử dụng toàn bộ 11 đặc trưng đề bài cung cấp

```
# ----- HÀM HỖ TRỢ MA TRẬN -----

def transpose(matrix):
    """Chuyển vị ma trận."""
    return [[row[i] for row in matrix] for i in range(len(matrix[0]))]

def zeros(shape):
    """Tạo ma trận zero với kích thước cho trước."""
    rows, cols = shape
    return [[0.0 for _ in range(cols)] for _ in range(rows)]

def identity(n):
    """Tạo ma trận đơn vị kích thước n."""
    I = zeros((n, n))
    for i in range(n):
        I[i][i] = 1.0
    return I

def multiply_matrices(A, B):
    """Nhân hai ma trận A và B."""
    A_rows, A_cols = len(A), len(A[0])
    B_rows, B_cols = len(B), len(B[0])
    if A_cols != B_rows:
        raise ValueError("Số cột của A phải bằng số hàng của B.")

    result = zeros((A_rows, B_cols))
    for i in range(A_rows):
        for j in range(B_cols):
            for k in range(A_cols):
                result[i][j] += A[i][k] * B[k][j]
    return result

def inverse_matrix(A):
    """Nghịch đảo ma trận bằng khử Gauss-Jordan."""
    n = len(A)
    # Tạo ma trận ghép [A | I]
    I = identity(n)
    aug = [A[i] + I[i] for i in range(n)]

    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(aug[r][i]))
        aug[i], aug[max_row] = aug[max_row], aug[i]

        pivot = aug[i][i]
        if abs(pivot) < 1e-10:
            raise ValueError("Ma trận không khả nghịch.")
        aug[i] = [x / pivot for x in aug[i]]

        for j in range(n):
            if i != j:
                aug[j] = [aug[j][k] - aug[i][k] * aug[j][i] for k in range(n)]
```

```

        factor = aug[j][i]
        aug[j] = [aj - factor * ai for aj, ai in zip(aug[j], aug[i])]

    inverse = [row[n:] for row in aug]
    return inverse

# ----- HỒI QUY OLS -----

def solve_ols(X, y):
    """Giải bài toán OLS:  $\theta = (X^T X)^{-1} X^T y$ """
    X_T = transpose(X)
    XTX = multiply_matrices(X_T, X)
    XTX_inv = inverse_matrix(XTX)
    XTy = multiply_matrices(X_T, y)
    theta = multiply_matrices(XTX_inv, XTy)
    return theta

def predict(X, theta):
    """Dự đoán y từ X và theta"""
    return multiply_matrices(X, theta)

def rss(y_true, y_pred):
    """Tính sai số bình phương trung bình"""
    n = len(y_true)
    total_error = 0.0
    for i in range(n):
        diff = y_true[i][0] - y_pred[i][0]
        total_error += diff ** 2
    return total_error

feature_names = df.drop(columns=['quality']).columns.tolist()

X_all_features = df.drop(columns=['quality']).values.tolist()
m = len(X_all_features)
X_a = [[1.0] + row for row in X_all_features]
y = [[val] for val in df['quality'].values.tolist()]

try:
    theta_a = solve_ols(X_a, y)
    y_pred_a = predict(X_a, theta_a)
    rss_a = rss(y, y_pred_a)

    print("--- Các hệ số (theta) của mô hình ---")
    print(f"Hệ số chặn (Intercept): {theta_a[0][0]:.4f}")
    for i, name in enumerate(feature_names):
        print(f"Hệ số cho {name}: {theta_a[i+1][0]:.4f}")
    print(f"\nSai số toàn phương (RSS) của mô hình: {rss_a:.4f}")

    from IPython.display import display, Math # chỉ dùng để hiện thị hàm y cho đẹp
    latex_str = f"\n=> y = {theta_a[0][0]:.4f} " + " ".join(
        [f"{'+' if theta_a[i+1][0]>=0 else '-'} {abs(theta_a[i+1][0]):.4f}x_{{{name}}}"
        ])
    display(Math(latex_str))

```

```
display(Math(latex_str))
```

```
except ValueError as e:
    print(f"Lỗi: {e}. Có thể do ma trận không khả nghịch.")
```

```

⇒ --- Các hệ số (theta) của mô hình ---
Hệ số chặn (Intercept): 42.9172
Hệ số cho "fixed acidity": 0.0475
Hệ số cho "volatile acidity": -1.0687
Hệ số cho "citric acid": -0.2687
Hệ số cho "residual sugar": 0.0350
Hệ số cho "chlorides": -1.5973
Hệ số cho "free sulfur dioxide": 0.0035
Hệ số cho "total sulfur dioxide": -0.0038
Hệ số cho "density": -39.4691
Hệ số cho "pH": -0.2456
Hệ số cho "sulphates": 0.7738
Hệ số cho "alcohol": 0.2694

```

Sai số toàn phương (RSS) của mô hình: 488.1765

✓ Câu b: Tìm và sử dụng duy nhất 1 đặc trưng tốt nhất (dùng Cross-Validation)

```

print("\n--- Cross-validation với từng đặc trưng đơn lẻ ---")

k_folds = 5
fold_size = m // k_folds
rss_scores_by_feature = {}

for i, feature_name in enumerate(feature_names):
    X_single_feature = [[row[i]] for row in X_all_features]
    X_b = [[1.0] + row for row in X_single_feature]

    current_rss = []

    for fold in range(k_folds):
        start = fold * fold_size
        end = (fold + 1) * fold_size if fold < k_folds - 1 else m

        X_val = X_b[start:end]
        y_val = y[start:end]

        X_train = X_b[:start] + X_b[end:]
        y_train = y[:start] + y[end:]

        try:
            theta_fold = solve_ols(X_train, y_train)
            y_pred_fold = predict(X_val, theta_fold)

            rss_fold = rss(y_val, y_pred_fold)
            current_rss.append(rss_fold)

```

```

except ValueError:
    current_rss.append(float('inf'))

avg_rss = sum(current_rss) / k_folds
rss_scores_by_feature[feature_name] = avg_rss
print(f"Đặc trưng {feature_name}: RSS trung bình = {avg_rss:.4f}")

# Tìm đặc trưng tốt nhất
best_feature_name = min(rss_scores_by_feature, key=rss_scores_by_feature.get)
best_feature_index = feature_names.index(best_feature_name)
best_rss = rss_scores_by_feature[best_feature_name]

print(f"\nĐặc trưng đơn lẻ tốt nhất: {best_feature_name} với RSS trung bình {best_rss:.4f}")

# Huấn luyện lại với đặc trưng tốt nhất
X_best = [[row[best_feature_index]] for row in X_all_features]
X_b_best = [[1.0] + row for row in X_best]

theta_best = solve_ols(X_b_best, y)
y_pred_best = predict(X_b_best, theta_best)
final_rss = rss(y, y_pred_best)

print("\n--- Mô hình cuối cùng với đặc trưng tốt nhất ---")
print(f"Hệ số chặn (Intercept): {theta_best[0][0]:.4f}")
print(f"Hệ số cho {best_feature_name}: {theta_best[1][0]:.4f}")
print(f"Sai số toàn phương (RSS): {final_rss:.4f}")

from IPython.display import display, Math # chỉ dùng để hiển thị hàm y cho đẹp (g
latex_str = f"\n=> y = {theta_best[0][0]:.4f} " + " ".join(
    [f"{'+' if theta_best[1][0]>=0 else '-'} {abs(theta_best[1][0]):.4f}x_{{{name}
}
)
display(Math(latex_str))

```



```

--- Cross-validation với từng đặc trưng đơn lẻ ---
Đặc trưng "fixed acidity": RSS trung bình = 162.4900
Đặc trưng "volatile acidity": RSS trung bình = 138.7001
Đặc trưng "citric acid": RSS trung bình = 154.8393
Đặc trưng "residual sugar": RSS trung bình = 163.7354
Đặc trưng "chlorides": RSS trung bình = 161.2292
Đặc trưng "free sulfur dioxide": RSS trung bình = 163.1559
Đặc trưng "total sulfur dioxide": RSS trung bình = 153.8624
Đặc trưng "density": RSS trung bình = 164.8234
Đặc trưng "pH": RSS trung bình = 162.6241
Đặc trưng "sulphates": RSS trung bình = 163.5490
Đặc trưng "alcohol": RSS trung bình = 119.0005

Đặc trưng đơn lẻ tốt nhất: "alcohol" với RSS trung bình 119.0005

--- Mô hình cuối cùng với đặc trưng tốt nhất ---
Hệ số chặn (Intercept): 1.7741
Hệ số cho "alcohol": 0.3747
Sai số toàn phương (RSS): 584.6750

```


✓ Câu c: Xây dựng một mô hình của riêng bạn cho kết quả tốt nhất

----- XÂY DỰNG CÁC HÀM HỖ TRỢ -----

```
def my_log(x):
    """Natural logarithm using Taylor series approximation"""
    if x <= 0:
        return float('-inf')
    if x == 1:
        return 0.0
    if 0.5 <= x <= 1.5:
        u = x - 1
        result = 0.0
        term = u
        for n in range(1, 50):
            result += term / n * (1 if n % 2 == 1 else -1)
            term *= u
            if abs(term / n) < 1e-10:
                break
        return result
    k = 0
    while x > 1.5:
        x /= 2.718281828
        k += 1
    while x < 0.5:
        x *= 2.718281828
        k -= 1

    return my_log(x) + k

def my_sqrt(x):
    """Square root using Newton's method"""
    if x < 0:
        return float('nan')
    if x == 0:
        return 0.0

    guess = x / 2.0
    for _ in range(20):
        new_guess = (guess + x / guess) / 2.0
        if abs(new_guess - guess) < 1e-10:
            break
        guess = new_guess
    return guess

def my_power(base, exp):
    """Power function using repeated multiplication"""
    if exp == 0:
        return 1.0
    if exp == 1:
        return base
```

```

    if exp == int(exp):
        exp = int(exp)
        result = 1.0
        if exp > 0:
            for _ in range(exp):
                result *= base
        else:
            for _ in range(-exp):
                result /= base
        return result
    else:
        return my_exp(my_log(base) * exp)

def my_exp(x):
    """Exponential function using Taylor series"""
    if x == 0:
        return 1.0

    result = 1.0
    term = 1.0
    for n in range(1, 50):
        term *= x / n
        result += term
        if abs(term) < 1e-10:
            break
    return result

def log_transform(values):
    """Apply logarithmic transformation to values"""
    result = []
    min_val = min(values)
    for val in values:
        if val <= 0:
            val = val - min_val + 1
        try:
            log_val = my_log(val)
            if log_val == float('-inf') or log_val != log_val:
                result.append(0.0)
            else:
                result.append(log_val)
        except:
            result.append(0.0)
    return result

def sqrt_transform(values):
    """Apply square root transformation"""
    result = []
    for val in values:
        if val < 0:
            result.append(-my_sqrt(abs(val)))
        else:
            result.append(my_sqrt(val))
    return result

```

```

def inverse_transform(values):
    """Apply inverse transformation (1/x)"""
    result = []
    for val in values:
        if abs(val) < 1e-10:
            result.append(1e10 if val >= 0 else -1e10)
        else:
            result.append(1.0 / val)
    return result

def polynomial_transform(X, degree):
    """Create polynomial features up to given degree"""
    n_samples = len(X)
    n_features = len(X[0])

    X_poly = [row[:] for row in X]

    for d in range(2, degree + 1):
        for i in range(n_samples):
            for j in range(n_features):
                X_poly[i].append(my_power(X[i][j], d))

    return X_poly

def cross_validate_model(X, y, k_folds=5):
    """Perform k-fold cross validation"""
    m = len(X)
    fold_size = m // k_folds
    rss_scores = []

    for fold in range(k_folds):
        start = fold * fold_size
        end = (fold + 1) * fold_size if fold < k_folds - 1 else m

        X_val = X[start:end]
        y_val = y[start:end]

        X_train = X[:start] + X[end:]
        y_train = y[:start] + y[end:]

        try:
            theta_fold = solve_ols(X_train, y_train)
            y_pred_fold = predict(X_val, theta_fold)
            rss_fold = rss(y_val, y_pred_fold)
            if rss_fold != rss_fold or rss_fold == float('inf'):
                rss_scores.append(1e10)
            else:
                rss_scores.append(rss_fold)
        except:
            rss_scores.append(1e10)

    return sum(rss_scores) / k_folds

def safe_transform_feature(feature_data, transform_func):
    """Safely apply transformation and handle errors"""

```

```

try:
    transformed = transform_func(feature_data)
    valid_transformed = []
    for val in transformed:
        if val != val or val == float('inf') or val == float('-inf'):
            valid_transformed.append(0.0)
        else:
            valid_transformed.append(val)
    return valid_transformed
except:
    return [0.0] * len(feature_data)

print("\n" + "="*60)
print("PHẦN C: XÂY DỰNG MÔ HÌNH TỐI ƯU")
print("="*60)

models_results = {}

# 1. Linear model (already done in part A)
print("\n1. Mô hình tuyến tính (11 đặc trưng):")
models_results['Linear (11 features)'] = rss_a
print(f"RSS = {rss_a:.4f}")

# 2. Single best feature (already done in part B)
print(f"\n2. Mô hình đơn đặc trưng tốt nhất ({best_feature_name}):")
models_results['Single best feature'] = final_rss
print(f"RSS = {final_rss:.4f}")

# 3. Test polynomial models with best feature
print("\n3. Mô hình đa thức với đặc trưng tốt nhất:")
best_feature_data = [row[best_feature_index] for row in X_all_features]

for degree in [2, 3, 4]:
    print(f"\n    Đa thức bậc {degree}:")
    X_poly = []
    for val in best_feature_data:
        row = [1.0, val]
        for d in range(2, degree + 1):
            row.append(my_power(val, d))
        X_poly.append(row)

    try:
        cv_score = cross_validate_model(X_poly, y)
        models_results[f'Polynomial degree {degree}'] = cv_score
        print(f"    RSS trung bình (CV) = {cv_score:.4f}")
    except:
        print(f"    Không thể tính toán mô hình bậc {degree}")

# 4. Transformation models with best feature
print("\n4. Mô hình biến đổi với đặc trưng tốt nhất:")
transformations = {
    'Square': lambda x: [val**2 for val in x],
    'Cubic': lambda x: [val**3 for val in x],
    'Square Root': sqrt_transform,

```

```

    'Inverse': inverse_transform,
    'Logarithmic': log_transform
}

for transform_name, transform_func in transformations.items():
    print(f"\n    {transform_name}:")

    try:
        transformed_data = safe_transform_feature(best_feature_data, transform_func)
        X_transform = [[1.0, val] for val in transformed_data]

        cv_score = cross_validate_model(X_transform, y)
        models_results[f'{transform_name} transform'] = cv_score
        print(f"    RSS trung bình (CV) = {cv_score:.4f}")
    except:
        print(f"    Không thể tính toán biến đổi {transform_name}")

# 5. Log-Log model (log(y) vs log(x))
print("\n5. Mô hình Log-Log:")
try:
    y_values = [val[0] for val in y]
    y_log = log_transform(y_values)
    x_log = safe_transform_feature(best_feature_data, log_transform)

    X_loglog = [[1.0, val] for val in x_log]
    y_loglog = [val for val in y_log]

    cv_score = cross_validate_model(X_loglog, y_loglog)
    models_results['Log-Log'] = cv_score
    print(f"RSS trung bình (CV) = {cv_score:.4f}")
except:
    print("Không thể tính toán mô hình Log-Log")

# 6. Log-Linear model (log(y) vs x)
print("\n6. Mô hình Log-Linear:")
try:
    X_loglin = [[1.0, val] for val in best_feature_data]
    y_loglin = [val for val in y_log]

    cv_score = cross_validate_model(X_loglin, y_loglin)
    models_results['Log-Linear'] = cv_score
    print(f"RSS trung bình (CV) = {cv_score:.4f}")
except:
    print("Không thể tính toán mô hình Log-Linear")

# 7. Linear-Log model (y vs log(x))
print("\n7. Mô hình Linear-Log:")
try:
    X_linlog = [[1.0, val] for val in x_log]

    cv_score = cross_validate_model(X_linlog, y)
    models_results['Linear-Log'] = cv_score
    print(f"RSS trung bình (CV) = {cv_score:.4f}")
except:
    print("Không thể tính toán mô hình Linear-Log")

```

```

# 8. Multiple feature polynomial model
print("\n8. Mô hình đa thức với nhiều đặc trưng:")
# Chọn 3 tính năng hàng đầu dựa trên hiệu suất của từng cá nhân
top_features = sorted(rss_scores_by_feature.items(), key=lambda x: x[1])[:3]
top_feature_indices = [feature_names.index(name) for name, _ in top_features]

print(f"    Sử dụng top 3 đặc trưng: {[name for name, _ in top_features]}")

# Tạo các đặc trưng đa thức với 3 đặc trưng hàng đầu
X_multi_poly = []
for row in X_all_features:
    new_row = [1.0] # intercept
    # Thêm số hạng tuyến tính
    for idx in top_feature_indices:
        new_row.append(row[idx])
    # Thêm số hạng bậc hai
    for idx in top_feature_indices:
        new_row.append(my_power(row[idx], 2))
    # Thêm số hạng bậc ba
    for idx in top_feature_indices:
        new_row.append(my_power(row[idx], 3))
    # Thêm các số hạng tương tác
    for i, idx1 in enumerate(top_feature_indices):
        for idx2 in top_feature_indices[i+1:]:
            new_row.append(row[idx1] * row[idx2])
    X_multi_poly.append(new_row)

try:
    cv_score = cross_validate_model(X_multi_poly, y)
    models_results['Multi-feature polynomial'] = cv_score
    print(f"    RSS trung bình (CV) = {cv_score:.4f}")
except:
    print("    Không thể tính toán mô hình đa đặc trưng")

# ----- KẾT QUẢ CUỐI CÙNG -----
print("\n" + "="*60)
print("KẾT QUẢ CUỐI CÙNG")
print("="*60)

print("\nBảng xếp hạng các mô hình (RSS thấp hơn = tốt hơn):")
sorted_models = sorted(models_results.items(), key=lambda x: x[1])

for i, (model_name, score) in enumerate(sorted_models, 1):
    print(f"{i:2d}. {model_name:25s}: RSS = {score:.4f}")

best_model_name, best_score = sorted_models[0]
print(f"\nMÔ HÌNH TỐI ƯU: {best_model_name}")
print(f"    RSS = {best_score:.4f}")

# Hiển thị chi tiết mô hình tối ưu
print("\n--- Chi tiết mô hình tối ưu ---")

if best_model_name == 'Linear (11 features)':
    print("Sử dụng mô hình tuyến tính với 11 đặc trưng (đã tính ở câu a)")

```

```

elif best_model_name == 'Single best feature':
    print(f"Sử dụng đặc trưng đơn lẻ: {best_feature_name}")
    print(f"y = {theta_best[0][0]:.4f} + {theta_best[1][0]:.4f} * {best_feature_n

else:
    # Các mô hình khác
    if 'Polynomial degree' in best_model_name:
        degree = int(best_model_name.split()[-1])
        X_final = []
        for val in best_feature_data:
            row = [1.0, val]
            for d in range(2, degree + 1):
                row.append(my_power(val, d))
            X_final.append(row)

        try:
            theta_final = solve_ols(X_final, y)
            equation = f"y = {theta_final[0][0]:.4f}"
            for d in range(1, degree + 1):
                coeff = theta_final[d][0]
                if d == 1:
                    equation += f" + {coeff:.4f}*x"
                else:
                    equation += f" + {coeff:.4f}*x^{d}"
            print(f"Phương trình: {equation}")
        except:
            print("Không thể hiển thị phương trình chi tiết")

    elif 'transform' in best_model_name:
        transform_name = best_model_name.replace(' transform', '')
        print(f"Sử dụng biến đổi {transform_name} cho đặc trưng {best_feature_nam

print("\n--- Kết luận ---")
print("Mô hình tối ưu đã được tìm thấy thông qua việc thử nghiệm nhiều phương phá
print("- Hồi quy tuyến tính đơn giản")
print("- Mô hình đa thức các bậc khác nhau")
print("- Các phép biến đổi: căn bậc hai, lập phương, nghịch đảo, logarithm")
print("- Mô hình Log-Log, Log-Linear, Linear-Log")
print("- Kết hợp đa đặc trưng với đa thức")
print("\nMô hình được chọn dựa trên Cross-Validation để đảm bảo khả năng tổng quá

```



RSS trung bình (CV) = 3.8663

7. Mô hình Linear-Log:

RSS trung bình (CV) = 118.6764

8. Mô hình đa thức với nhiều đặc trưng:

Sử dụng top 3 đặc trưng: ['"alcohol"', '"volatile acidity"', '"total sulf
RSS trung bình (CV) = 121.0422

KẾT QUẢ CUỐI CÙNG

Bảng xếp hạng các mô hình (RSS thấp hơn = tốt hơn):

1. Log-Log	: RSS = 3.8540
2. Log-Linear	: RSS = 3.8663
3. Polynomial degree 3	: RSS = 117.0551
4. Polynomial degree 4	: RSS = 117.2399
5. Logarithmic transform	: RSS = 118.6764
6. Linear-Log	: RSS = 118.6764
7. Inverse transform	: RSS = 118.7238
8. Square Root transform	: RSS = 118.7895
9. Polynomial degree 2	: RSS = 118.8343
10. Square transform	: RSS = 119.7335
11. Cubic transform	: RSS = 120.9038
12. Multi-feature polynomial	: RSS = 121.0422
13. Linear (11 features)	: RSS = 488.1765
14. Single best feature	: RSS = 584.6750

MÔ HÌNH TỐI ƯU: Log-Log

RSS = 3.8540

--- Chi tiết mô hình tối ưu ---

--- Kết luận ---

Mô hình tối ưu đã được tìm thấy thông qua việc thử nghiệm nhiều phương pháp:

- Hồi quy tuyến tính đơn giản
- Mô hình đa thức các bậc khác nhau
- Các phép biến đổi: căn bậc hai, lập phương, nghịch đảo, logarithm
- Mô hình Log-Log, Log-Linear, Linear-Log
- Kết hợp đa đặc trưng với đa thức

Mô hình được chọn dựa trên Cross-Validation để đảm bảo khả năng tổng quát hóa

✓ =====

MÔ TẢ Ý TƯỞNG THỰC HIỆN VÀ CÁC HÀM

=====

=====

Mô tả ý tưởng thực hiện

Ý tưởng cốt lõi của đề án này là xây dựng và tối ưu hóa một mô hình hồi quy tuyến tính để dự đoán chất lượng rượu vang. Quá trình thực hiện không chỉ dừng lại ở việc áp dụng một công thức duy nhất mà là một cuộc khảo sát có hệ thống để tìm ra mô hình hiệu quả nhất, dựa trên nguyên tắc cơ bản của phương pháp Bình phương tối thiểu (OLS).

Quá trình được chia thành ba giai đoạn chính:

1. Xây dựng mô hình cơ sở (Câu a):

- Ý tưởng là thiết lập một "đường cơ sở" (baseline) để so sánh. Mô hình này sử dụng toàn bộ 11 đặc trưng hóa học mà dữ liệu cung cấp.
- Bằng cách áp dụng trực tiếp công thức nghiệm của OLS (Normal Equation), chúng ta có được một mô hình ban đầu và một giá trị sai số (RSS). Kết quả này cho thấy hiệu suất tổng quan khi tất cả các thông tin đều được xem xét, nhưng có thể bị nhiễu bởi các đặc trưng không liên quan hoặc có mối tương quan phức tạp với nhau.

2. Sàng lọc đặc trưng quan trọng nhất (Câu b):

- Nhận thấy mô hình với tất cả 11 đặc trưng có thể không phải là tối ưu, ý tưởng tiếp theo là xác định xem liệu có một đặc trưng đơn lẻ nào có sức mạnh dự đoán vượt trội hay không.
- Để thực hiện điều này một cách khách quan, kỹ thuật **Kiểm định chéo K-Fold (K-Fold Cross-Validation)** được áp dụng. Thay vì chỉ huấn luyện và kiểm tra trên cùng một tập dữ liệu (dễ dẫn đến học vẹt), phương pháp này đánh giá khả năng tổng quát hóa của mỗi đặc trưng trên nhiều tập dữ liệu con khác nhau.
- Đặc trưng nào cho sai số RSS trung bình thấp nhất qua các lần kiểm định chéo sẽ được coi là "đặc trưng tốt nhất". Kết quả từ bước này ("alcohol") trở thành nền tảng cho giai đoạn tối ưu hóa tiếp theo.

3. Thử nghiệm và Tối ưu hóa mô hình (Câu c):

- Đây là giai đoạn sáng tạo nhất, với ý tưởng rằng mối quan hệ giữa các đặc trưng và chất lượng rượu vang có thể không hoàn toàn tuyến tính. Mục tiêu là tìm ra một mô hình cho kết quả tốt nhất bằng cách khám phá các phương pháp biến đổi và kết hợp đặc trưng.
- **Hồi quy đa thức (Polynomial Regression):** Thử nghiệm xem việc thêm các bậc cao hơn (bậc 2, bậc 3) của đặc trưng tốt nhất ("alcohol") có giúp mô hình "uốn cong" theo dữ liệu tốt hơn không.
- **Biến đổi đặc trưng (Feature Transformation):** Áp dụng các phép biến đổi toán học phổ biến (logarit, căn bậc hai, nghịch đảo) lên đặc trưng tốt nhất. Ý tưởng là các phép biến đổi này có thể "tuyến tính hóa" mối quan hệ giữa đặc trưng và biến mục tiêu, giúp mô hình hồi quy tuyến tính hoạt động hiệu quả hơn.

- **Biến đổi biến mục tiêu và đặc trưng (Log-Log, Log-Linear):** Khám phá các dạng mô hình kinh tế lượng cổ điển bằng cách biến đổi cả biến đầu vào X và biến đầu ra y . Ví dụ, mô hình Log-Log ($\log(y) \sim \log(x)$) rất hữu ích trong việc mô hình hóa các mối quan hệ có tính co giãn (elasticity).
- **Kết hợp nhiều đặc trưng có chọn lọc:** Thay vì chỉ dùng 1 hoặc tất cả 11 đặc trưng, ý tưởng cuối cùng là chọn ra một vài đặc trưng hàng đầu (dựa trên kết quả từ câu b) và xây dựng một mô hình đa thức phức tạp hơn, bao gồm cả các số hạng tương tác (ví dụ: $\text{alcohol} * \text{sulphates}$) để nắm bắt mối quan hệ kết hợp giữa chúng.

Xuyên suốt quá trình, **Cross-Validation** luôn được sử dụng làm thước đo chính để đánh giá và so sánh các mô hình, đảm bảo rằng mô hình được chọn không chỉ tốt trên dữ liệu đã thấy mà còn có khả năng dự đoán chính xác trên dữ liệu mới.

Mô tả các hàm

Để thực hiện đồ án mà không phụ thuộc vào các thư viện tính toán cấp cao như `numpy` hay `scikit-learn`, các hàm toán học và thuật toán cốt lõi đã được tự xây dựng từ đầu. Các hàm này có thể được chia thành bốn nhóm chính:

Nhóm 1: Các hàm thao tác ma trận

Đây là những công cụ nền tảng để thực hiện các phép tính đại số tuyến tính.

- `transpose(matrix)`: Nhận một ma trận và trả về ma trận chuyển vị của nó.
- `zeros(shape)`: Tạo một ma trận chứa toàn số 0 với kích thước (`rows`, `cols`) được chỉ định.
- `identity(n)`: Tạo ra ma trận đơn vị cấp n (ma trận vuông có đường chéo chính là 1, các vị trí khác là 0).
- `multiply_matrices(A, B)`: Thực hiện phép nhân giữa hai ma trận A và B , trả về ma trận kết quả.
- `inverse_matrix(A)`: Tìm ma trận nghịch đảo của một ma trận vuông A bằng phương pháp khử Gauss-Jordan. Đây là hàm phức tạp và quan trọng nhất trong nhóm này, cần thiết cho việc giải phương trình pháp tuyến.

Nhóm 2: Các hàm cốt lõi của Hồi quy OLS

Nhóm này trực tiếp triển khai thuật toán hồi quy tuyến tính.

- `solve_ols(X, y)`: Hàm trung tâm của mô hình. Nó nhận vào ma trận đặc trưng X (đã bao gồm cột hệ số chặn) và vector mục tiêu y , sau đó giải bài toán OLS bằng công thức nghiệm $\theta = (X^T X)^{-1} X^T y$ bằng cách sử dụng các hàm ma trận ở trên. Hàm này trả về vector hệ số θ .
- `predict(X, theta)`: Dùng bộ hệ số θ đã tính được để dự đoán giá trị đầu ra $\hat{y}$ cho một tập dữ liệu đầu vào X . Phép tính đơn giản là $\hat{y} = X\theta$.

- `rss(y_true, y_pred)` : Tính toán Tổng bình phương các phần dư (Residual Sum of Squares), là độ đo sai số chính được sử dụng trong toàn bộ đồ án để đánh giá hiệu suất của mô hình.

Nhóm 3: Các hàm xử lý đặc trưng và đánh giá mô hình

Nhóm này phục vụ cho việc kỹ thuật đặc trưng (feature engineering) và đánh giá mô hình một cách khoa học.

- `cross_validate_model(X, y, k_folds=5)` : Triển khai quy trình kiểm định chéo K-fold. Nó tự động chia dữ liệu thành K phần, lần lượt huấn luyện trên K-1 phần và đánh giá trên phần còn lại, cuối cùng trả về sai số RSS trung bình.
- `log_transform(values)` , `sqrt_transform(values)` , `inverse_transform(values)` : Áp dụng các phép biến đổi toán học tương ứng (logarit, căn bậc hai, nghịch đảo) lên một vector dữ liệu.
- `polynomial_transform(X, degree)` : Tạo ra các đặc trưng đa thức từ các đặc trưng gốc lên đến bậc `degree` .
- `safe_transform_feature(...)` : Một hàm tiện ích để áp dụng các hàm biến đổi một cách an toàn, xử lý các trường hợp ngoại lệ (ví dụ `log(0)`) để tránh làm chương trình bị lỗi.

Nhóm 4: Các hàm toán học cơ bản tự triển khai

Để hoàn toàn độc lập, các hàm toán học cơ bản cũng được tự xây dựng.

- `my_log(x)` , `my_exp(x)` : Tính logarit tự nhiên và hàm mũ e^x bằng cách khai triển chuỗi Taylor.
- `my_sqrt(x)` : Tính căn bậc hai bằng phương pháp lặp của Newton.
- `my_power(base, exp)` : Tính lũy thừa $base^{exp}$, hỗ trợ cả số mũ nguyên và thực.